

东南大学专业学位型研究生 学位论文开题报告及论文实施计划

院（系、所） 网络空间安全学院

学位类别 专业硕士

专业领域 电子信息

研究生姓名 张景耀

指导教师（校内） 徐坚皓

指导教师（校外） 无

开题报告日期 2026.5

填表须知

1、论文开题报告由研究生本人向审议小组报告并听取意见后，由研究生本人填写此表。

2、论文开题报告填写完成后，必须经导师审批，通过后方能提交。

3、博士生应在第四学期内、硕士生应在第三学期内完成此开题报告。开题报告经研究生秘书在网上审核确认（硕士生至少半年、博士生至少一年）后方可申请答辩。

4、研究生开题前应填写查新报告。查新报告对专业学位博士作为必要环节。博士生查新工作可委托图书馆负责，也可在完成网络文献检索类研究生课程的学习或参加学校组织的网络文献检索培训后，自行组织查新检索，自行组织查新需要详细文献查新述评作为附件。自行查新报告须经导师审查后由开题报告审核专家组审核签字（或盖章）。硕士生开题查新参考上述办法，不作硬性要求。

5、本表一式两份，一份研究生自留放入本人“研究生档案材料袋”；一份由院（系、所）保存并归入院（系、所）研究生教学档案。

6、学位类别为：工程硕士；公共管理硕士；法律硕士（非法学）；工商管理硕士；建筑学硕士；风景园林硕士；临床医学硕士；公共卫生硕士；旅游管理硕士；会计硕士；国际商务硕士；资产评估硕士；工程管理硕士；艺术硕士；工程博士；医学博士等。

7、本表下载区：<http://seugs.seu.edu.cn/3676/list.htm>。本表电子文档打印时用 A4 纸张，格式不变，内容较多可以加页。

一、学位论文开题报告

论 文 题 目	基于 LLM 约束求解的编译器防御机制模糊测试方法研究								
研 究 方 向	网络空间安全								
题 目 来 源	国家	部委	省	市	厂、矿	自选	有无合同	经费数	备注
						√			
题 目 类 型	理论 研究	应用 研究	工程 技术	跨学科 研 究	其他				
开题报告内容（具体要求见《东南大学研究生论文选题和开题报告的原则和要求》） 一、研究背景 CVE-2023-4039 暴露了 GCC 在 AArch64 上栈保护机制的一处长期失效。当函数使用变长数组（VLA）或 <code>alloca</code> 时，编译器把栈金丝雀放在 VLA 之上、保存的返回地址却落在 VLA 之下，越界写入可以先于金丝雀检查改写返回地址，<code>-fstack-protector</code> 因此形同虚设——这种 buggy 状态下的栈帧布局可参见图 1。编译过程不报错，二进制能通过功能测试，缺陷只在攻击者构造特定输入时才显现。该漏洞自 GCC 早期版本即已存在，直到 2023 年才由外部研究者系统分析并修复。问题的共性也由此显现：编译器侧的防御机制依赖前端识别、中端 pass 与后端代码生成共同维护一组安全不变量，其中任何一处与具体 ISA 的栈布局、寄存器分配或指令编码假设不一致，防御机制就会悄然失效——对外仍然“存在”，对攻击者不再有效。									



图 1 CVE-2023-4039 buggy 栈帧布局

编译器所承载的防御机制数量本身已相当庞大：栈金丝雀、`_FORTIFY_SOURCE`、Intel CET 中的 IBT、AArch64 的 BTI 与 PAC、Shadow Stack、SafeStack、KCFI、CFI、HCFR 等，分别保护栈完整性、控制流、对象边界，且仍在扩展。目标 ISA 方面，x86-64、AArch64、RISC-V、LoongArch、MIPS、PowerPC 等架构在调用约定、寄存器分配与指令编码上差异显著，每种机制在各 ISA 上对应若干份独立的后端模板与中端 pass 实现。二者构成“防御机制 × ISA”的二维研究空间（见图 2），每一格对应一组需联合维护的安全不变量，目前已被覆盖的格点仍只是其中很小一部分。

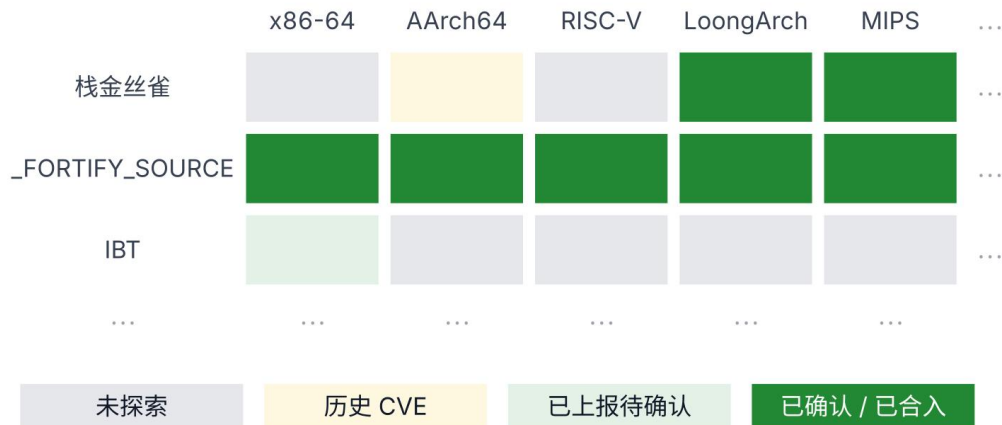


图 2 防御机制 $\times$ ISA 二维研究空间

这个矩阵的规模使人工审计难以全面覆盖。与 CVE-2023-4039[16] 同源的 PR-96191[7]（AArch64 / ARM 上栈金丝雀寄存器残留）2020 年修复时，就未涉及 MIPS、LoongArch、Xtensa、CSKY 等 fallback 后端；单点防御机制上的实现疏忽，往往同时存在于矩阵中其他格点而长期不被发现。本课题初步实验阶段已对若干格点做了试探性验证，向 GCC 上游报告并确认了多例同类形态的真实漏洞[17-20]，第五章将给出具体案例。

上述失效有一个共同特征：编译过程不出错、二进制正常运行，缺陷仅在特定输入下显现——即“静默失效（silent failure）”。Csmith、YARPPGen 等以差分输出或编译器崩溃为信号的工具对此类缺陷几乎不敏感；覆盖率引导的模糊测试虽然可以把执行推进到相关代码路径，但缺少判断该路径是否仍满足安全契约的预言机。问题由此浮现：编译器是绝大多数软件防御机制的实际载体，矩阵中的静默失效直接削弱整个软件生态的运行时安全保障，而现有自动化测试方法尚未把“防御机制的安全契约”纳入判定体系。如果能把矩阵格点上的不变量翻译为可测断言，并主动引导编译器走到对应代码路径，就既可覆盖既有缺陷家族的同类变种，也有望在后续优化引入同类回归时及早暴露。本研究以此为出发点，结合大语言模型的代码理解能力与覆盖率反馈，构建一条针对编译器防御机制静默失效的模糊测试方法。

二、研究现状

编译器模糊测试的研究可以从两个角度来看：一是预言机（oracle），决定什么算缺陷；二是种子变异方法，决定测试输入怎么产生。早期工作以崩溃或输出差异为信号、以随机或文法规则生成种子；覆盖率引导与大语言模型的引入分别使二者向目标导向和语义理解方向演进。下文沿这两个维度梳理现状。

2.1 编译器模糊测试预言机

预言机回答"什么样的执行结果算缺陷", 现有工作大致可分五类。

崩溃预言机[1,3,4]以编译期或运行期的崩溃 (ICE、断言、segfault、超时) 为信号, 实现简单, 是 AFL、libFuzzer 迁移到编译器后的默认选择。差分预言机[1,2]对若干编译器 (或同一编译器不同优化等级) 就同一程序的运行结果做交叉比对, 差异即为 mis-compile; Csmith、YARPGen 围绕"避免未定义行为"这一前提大规模生成 C 程序来驱动差分。变形测试预言机以 Le 等提出的 EMI[11] 为代表: 对同一输入下未执行的代码做删改, 新程序应产生相同结果, 差异即为 mis-compile, 后续工作继续在 live region 等价变换上探索。Sanitizer 预言机借助 ASan、UBSan、MSan 把内存破坏与未定义行为转为显式信号; UBfuzz[12] 进一步用 crash-site mapping 反向定位 sanitizer 自身实现缺陷。还有面向子领域的复合预言机, 如 HirGen[13] 在 IR 层同时使用差分与变形来提高对深度学习编译器深层优化错误的敏感度。

这些预言机的判据都建立在"崩溃"或"可观察的输出差异"之上。当缺陷的形态是"代码正常编译运行, 但运行时保护已被静默削弱"时, 它们无法触发。

2.2 传统种子变异方法

传统路线上, 覆盖率作为推进信号已是主流。Csmith[1] 与 YARPGen[2] 基于概率文法从空白合成不含未定义行为的 C 程序, 再靠差分预言机发现 mis-compile。LangFuzz 类工作记录种子语法树后做子树替换式的 grammar-aware 变异, 多用于 JavaScript 引擎。AFL、libFuzzer[3,4] 以字节级覆盖率反馈为导向, 被广泛用于编译器的解析与优化组件, 但难以保证生成程序的语法和语义合法性。GrayC[14] 把覆盖率引导与一组保持语义合法性的源码级变异 (表达式替换、控制结构改写) 相结合, 专门面向 C 编译器与分析器, 覆盖率和缺陷发现率均优于纯生成式工具。EMI 系列[11]在已有种子的未执行区域做语义保持变换, 可视为变异与变形预言机的组合。这条路线的局限在于: 变异规则由人工先验定义, 覆盖率只提供"是否走入新代码"的近似进度信号, 变异本身缺乏对目标代码语义的理解。

2.3 大模型驱动的种子变异方法

大模型在代码理解与生成上的能力近年被引入以缓解传统变异在质量上的瓶颈。WhiteFox[5] 从编译器优化 pass 源码出发, 先用大模型归纳触发该优化的输入模式 (requirement summarization), 再据此生成测试用例, 使种子生成具备对编译器内部逻辑的感知; 该方法在 PyTorch、TVM 等深度学习编译器上发现了若干优化路径深处的 bug。HLPFuzz[15] 进一步把大模型与覆盖率反馈耦合: 在灰盒主循环之外引入白盒求解器, 按"未覆盖且后继最多"的策略选取目标基本块, 把单基本块翻译为约束求解任务交由大模型完

成，并通过渐进式上下文构建缓解注意力衰减。这些工作表明大模型可以充当语义感知的变异引擎，但它们的目标设定仍是常规可达性——"让程序走到某个未覆盖位置"，并不关心目标路径所承担的安全契约。

综合来看，针对编译器自身防御机制的自动化测试在公开文献中仍然稀少：少数工作面向单一机制做差分测试（如扫描 IBT landing pad 存在性），另一些工作从二进制层面做事后审计，但难以反向定位到引入缺陷的编译器代码路径。本课题在变异方法上沿用大模型与覆盖率联合驱动的思路，但把推进目标限定在"防御机制相关代码"；在预言机上把机制不变量翻译为 fuzz 框架可直接使用的判定接口，使方法对静默失效保持敏感。

三、研究目标及研究内容

3.1 研究目标

本课题要构建一种面向编译器自身防御机制的模糊测试方法，并在 GCC 与 LLVM 上完成端到端验证。与以崩溃或差分输出为信号的传统编译器 fuzzer 不同，本方法以"防御机制是否仍满足其安全契约"为判据，自动暴露编译器仍能正常生成代码、但运行时保护已被静默削弱的失效。具体目标如下：(a) 形成一套覆盖主流防御机制的不变量集合，每条不变量具备可由二进制或运行行为判定的证据形式，在 GCC 与 LLVM 上保持一致的描述结构；(b) 构建一个可同时面向 GCC 与 LLVM、跨架构运行的 fuzzer 原型，以防御机制相关代码的源代码覆盖率为推进信号，用大模型完成种子精细变异；(c) 通过该原型在 GCC 与 LLVM 上发现可被上游确认的真实缺陷，验证方法有效性与跨编译器迁移性。三个目标依次推进，其依赖关系与对应的工作内容在 3.2 节中呈现（见图 3）。

3.2 研究内容

本课题分为以下三部分（见图 3），彼此存在依赖关系：不变量形式化是预言机与种子目标选择的事实基础；预言机为 fuzz 主循环提供安全判定接口；覆盖率驱动的种子生成则负责把 fuzzing 推进到能触发不变量的代码路径上。

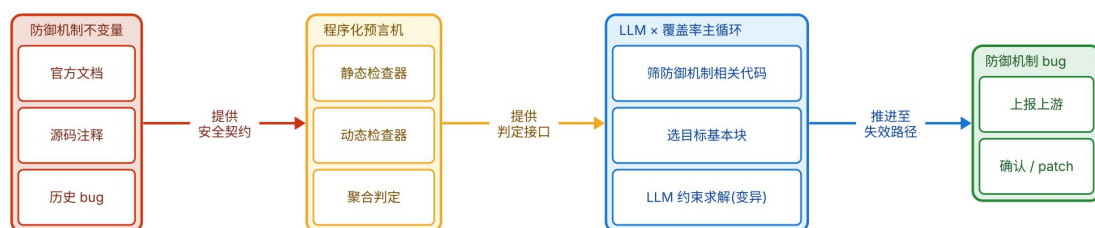


图 3 研究内容工作链条

(1) 防御机制不变量的调研与形式化。针对 `stack canary`、`_FORTIFY_SOURCE`、`IBT` 等机制，从三类来源提炼其安全不变量：一是编译器源码中的注释与断言，如 GCC 中 `AArch64` 后端关于 `canary` 槽位置关系的安全注释、`ix86` 关于 `ENDBR` 立即数禁止的谓词，以及 LLVM 中 `lib/CodeGen/StackProtector` 与 `X86InstrInfo` 等模块的对应实现；二是 GCC、LLVM 自身文档（`Clang Command Line Reference`、`Attribute Reference`、`CFI Design` 等）以及 `glibc` 与 ABI 文档对开关 `flag`、`_chk` 接口、`CET ELF property` 等的规定；三是已确认的历史 bug 与对应 patch——从修补点反推被违反的不变量。每条不变量按统一字段记录陈述、所属编译器与版本、证据片段、信息源及版本敏感性，便于机械检索与跨版本回归。

(2) 不变量到预言机的工程化落地。把 (1) 中的不变量翻译为可在二进制层面或运行行为上判定的检查器。框架将检查器分为静态与动态两类：静态检查通过 ELF 符号、指令流模式、属性 `note` 做断言式判定；动态检查通过受控执行（如对 `buffer` 填充长度做二分搜索）观察运行时行为。预言机判据完全建立在二进制和运行行为之上，与产生该二进制的具体编译器解耦，GCC 与 LLVM 产物共用同一套检查器。每个机制配置一个聚合器，任一检查器返回 `Fail` 即记机制级违例并附带证据；同时为每个机制记录开机制 / 关机制两套 `flag profile`，框架据此对同一颗种子做对照判定，避免与编译器自身缺省行为混淆。

(3) 大模型与覆盖率联合驱动的种子生成。以 GCC 与 LLVM 中防御机制相关代码的源代码覆盖率为目标信号（GCC 侧 `gcov`，LLVM 侧 `SanitizerCoverage / source-based coverage`），结合控制流图按“未覆盖且与防御机制相关性高”的策略选取基本块作为约束求解目标，相关性由 CFG 后继分支数与所在代码区域的防御机制权重共同决定。每轮求解的流程：构造目标上下文（目标函数源码、目标基本块标注、覆盖前驱的种子作为示例），调用大模型生成新 C 种子，编译产物再经覆盖率与预言机检查；结果分析阶段同时看覆盖增量、是否命中目标、是否触发 bug 三个信号，任一为真即把种子纳入语料库。种子未命中目标时，通过轻量级调用追踪定位与基线种子的发散点，把发散函数源码作为额外上下文反馈给大模型进入下一轮变异。整个过程把覆盖率推进与安全契约判定统一在同一条主循环中，大模型的生成始终被防御机制相关代码路径所牵引。

四、技术路线

技术路线分为总体架构、不变量预言机框架、大模型约束求解主循环、跨架构与编译选项调度四部分，对应研究内容 (1)–(3) 的工程实现。GCC 上的初步验证证据见第五章。

1. 总体架构

系统以一条 `fuzz` 主循环为骨架，由 LLM 变异、编译器、预言机三个子系统加种子语料

库、配置模块共同支撑（总体结构见图 4）。Fuzz 引擎承担主循环调度与提示词组装两项职能，把当前覆盖率、目标基本块和已有种子拼装成提示词后交给 LLM 变异子系统调用远端模型，再把返回的新种子分发给后续环节。编译器子系统封装预先插桩的 GCC 与 LLVM 构建：GCC 侧通过 gcov 采集 .gcda 覆盖率数据，LLVM 侧通过 SanitizerCoverage 与 source-based coverage 采集 .profrac 数据，二者经统一接口归并为同构的"行级覆盖 → 种子 ID"映射，主循环对底层编译器无感知。预言机子系统按防御机制聚合不变量检查器，对每颗成功编译的种子做安全契约判定，判据建立在 ELF 二进制与运行行为之上，GCC 与 LLVM 产物共用同一套检查器。种子语料库持久化满足质量阈值的种子并维护家系信息。配置模块按"防御机制·ISA·flag"三元组下发任务：把工具链选择（GCC / LLVM）与对应 flag 交给编译器子系统，把目标 ISA 对应的 QEMU 工具链交给预言机的执行层，使同一份种子能在不同后端上做对照实验。

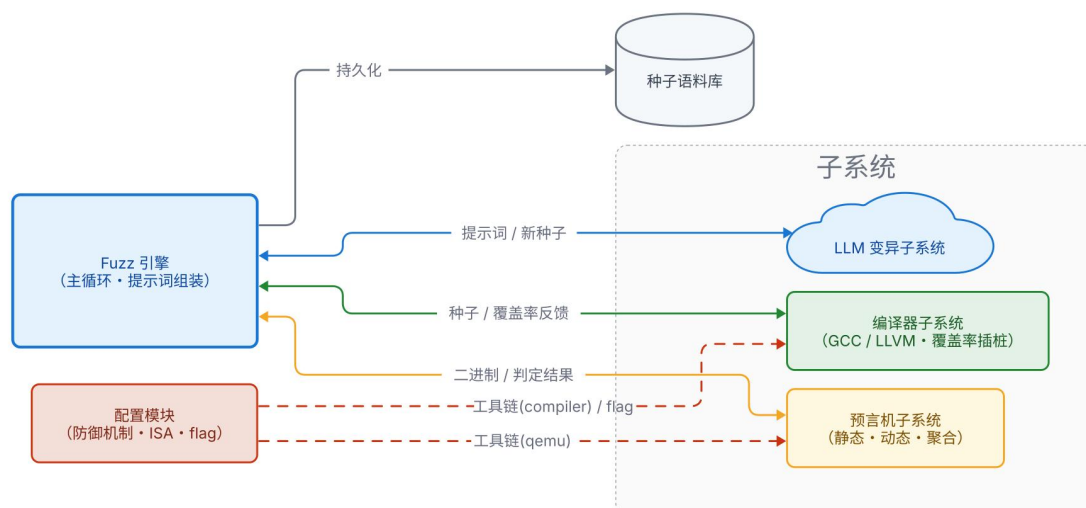


图 4 DeFuzz 总体架构

2. 防御机制不变量预言机框架

每个机制对应一个聚合器，下挂若干检查器，输入端把同一颗种子的源码、汇编与最终二进制一并交给检查器（判定流程见图 5）。检查器分为静态与动态两类：静态检查器走汇编分析与字节检查路径，读 ELF 符号表、指令字节模式与 .note 段做断言式判定，例如确认间接跳转目标处的 ENDBR 字节、_chk 接口被正确链接；动态检查器以插桩检测和二分搜索为主要手段，通过受控执行观察运行时行为，典型做法是对溢出 buffer 的填充长度做二分搜索，定位 canary 触发的实际边界。每个检查器返回 Pass、Fail、Error 三态，Error 仅用于隔离基础设施故障，不参与机制级判定；聚合时任一 Fail 即视为机制级违例，并把触发证据回

写到种子记录。对易与编译器缺省行为混淆的检查器，框架同时跑开机制与关机制两套 flag profile，比对两次判定的差异，确保 Pass / Fail 反映的是机制本身。预言机维护一份元信息缓存，复用二分搜索结果与动态符号查询，避免同一颗二进制被多次解析或执行。

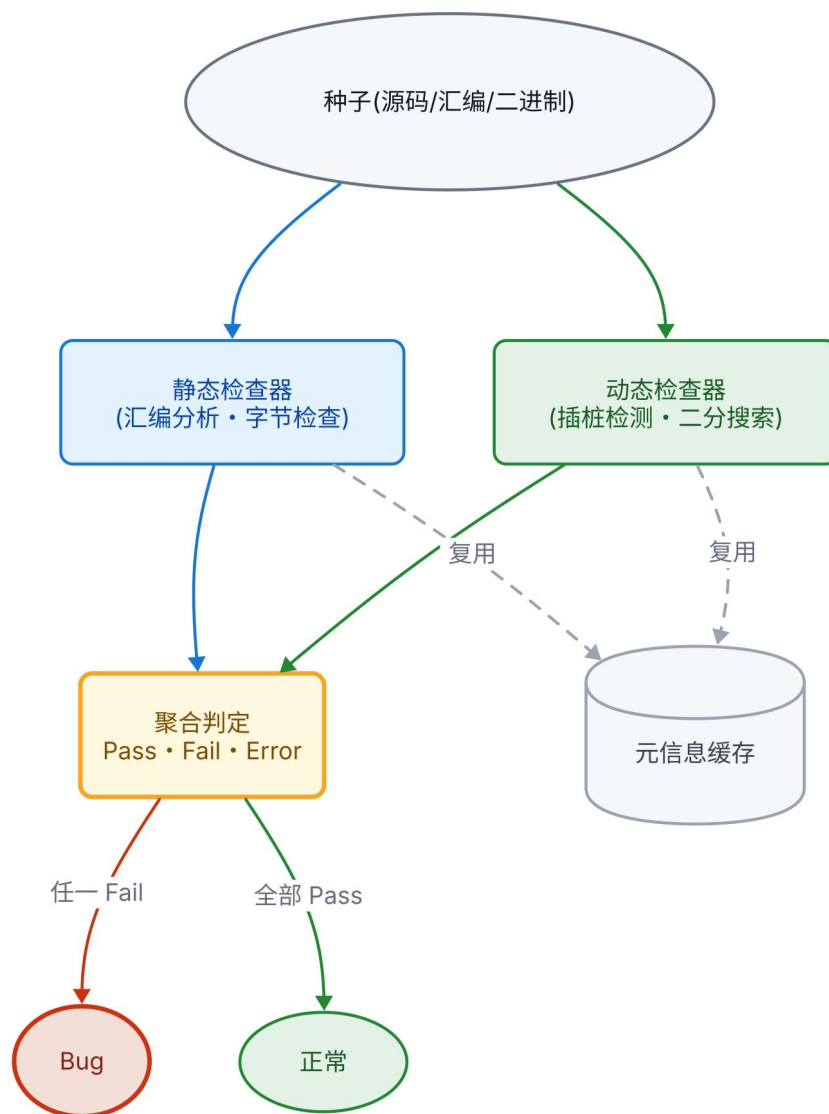


图 5 防御机制不变量预言机判定流程

3. 大模型约束求解主循环

主循环输入为当前覆盖率映射与编译器源码控制流图（迭代过程见图 6）。每轮先在未覆盖基本块中按相关性挑出一个目标，相关性由该基本块在 CFG 上的后继分支数和所属代码区域对防御机制的关联程度共同决定，使搜索集中在尚未触达且与防御逻辑相关的代码上；随后构造目标上下文：目标函数源码片段（来自当前所测的 GCC 或 LLVM 版本）、逐行覆盖标注、目标基本块位置，以及覆盖目标前驱的若干种子作为示例。提示词组装层把上

下文拼接到基础约束求解提示词与机制契约要求之后，提交给大模型生成新 C 种子。编译失败时把错误信息与上一版源码反馈进入下一轮；编译成功后覆盖率分析器判断是否命中目标，预言机给出安全判定。结果分析阶段同时看覆盖增量、是否命中目标、是否触发 bug 三个信号，任一为真即把该种子纳入语料库。种子未命中目标但带来新覆盖时，轻量级动态追踪定位它与基线种子在调用序列上的首个发散点，把发散函数源码作为额外上下文回喂提示词组装层，再次请求模型做精细变异。目标基本块全部覆盖后，主循环可切换到随机变异阶段，仅以预言机为持久化条件继续探索。

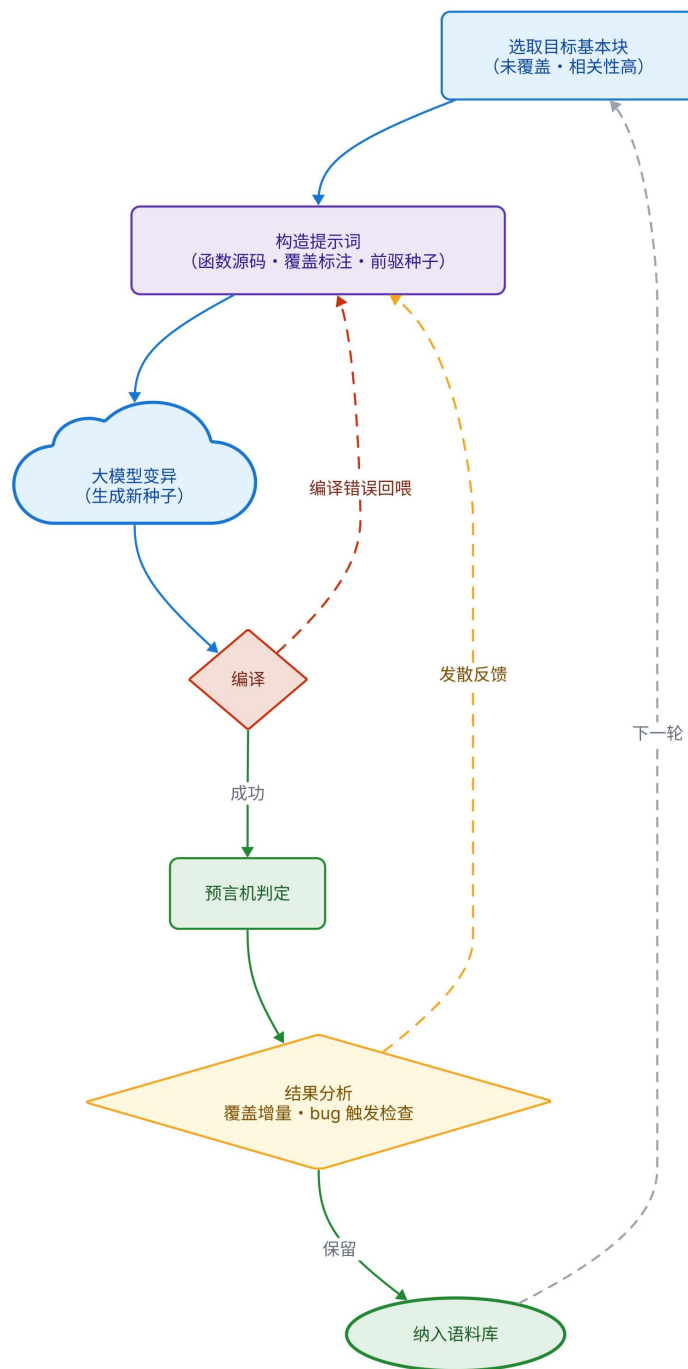


图 6 大模型约束求解主循环

4. 跨架构与编译选项调度

配置模块把编译命令参数化为编译器（GCC / LLVM）、ISA、架构相关 flag、策略相关 flag profile 四元组，向编译器子系统下发工具链与 flag，向预言机执行层下发对应 ISA 的 QEMU 工具链，同一份种子据此分发给不同后端，目标二进制通过 QEMU 用户态执行。flag profile 同时包含开机制（如 -fstack-protector-strong）与关机制（如 -fno-stack-protector）两套，预言机据此对同一颗种子做对照判定；关机制产物仅用于交叉验证，不进入主语料库。GCC 与 LLVM 在防御机制 flag 拼写上的差异（如 -fcf-protection 对 -fsanitize=cfi）由 FlagScheduler 统一抽象为"机制级"配置，向上层屏蔽。BinaryInspector 屏蔽 ISA 差异：通过 Go 标准库读取 ELF 符号表与 .note 段，为检查器提供导入函数列表、ENDBR 字节模式扫描、栈布局采样等能力。

五、初步实验

目前已在 GCC 多个版本（若干稳定版与一个本地构建的 master 快照）上对栈金丝雀、_FORTIFY_SOURCE、IBT 等部分防御机制做了实验，LLVM 侧同类实验纳入下一阶段。已完成实验共发现 3 例可被上游确认的 silent failure[17-20]；下文展开其中两例，第三例（栈金丝雀在多个 ISA 的 fallback 后端上函数返回时寄存器未清零，与 PR-96191[7] 同源）篇幅较短，在 5.3 节简述。

5.1 案例一：_FORTIFY_SOURCE 优化改写后运行时上限被静默放宽

_FORTIFY_SOURCE 是 GCC 与 glibc 配合的源代码加固机制。开启后，glibc 头文件在预处理阶段把 strcat 等危险接口重写为带 size 参数的 __strcat_chk，运行时据此检查目的缓冲区上限。GCC 的 strlen pass 进一步识别已知前缀长度的目的指针，将 __strcat_chk 改写为性能更优的 __stpcpy_chk，并把目的指针前移 strlen(dst) 字节。这里的安全不变量是：目的指针前移后，传给 __stpcpy_chk 的 size 参数必须由原始 objsize 减去 strlen(dst)，否则运行时上限就被静默放宽 strlen(dst) 字节。

实测中，本方法构造的种子在该路径上稳定触发了 size 参数未相应减小的实例。反汇编层面 strcat → __strcat_chk → __stpcpy_chk 整条链条仍然出现，看似 FORTIFY 仍在生效，但"按 dst 剩余空间收紧上限"那一步已被丢弃。该缺陷自 GCC 4.7 起存在，跨越十余年未被发现；上报后已被确认（PR-125079[17]）并合并入主分支，预计随 GCC 16.1 发布，开发团队同时计划向 GCC 13、14、15 回送补丁。这条路径与 2018 年的 PR-87672[6] 处于同一文件：当年的修复只覆盖了一个对称分支，另一支被遗漏至今——这本身就是第一章所述"矩阵同源

遗漏"的又一例证。

预言机方面，本例命中"FORTIFY 调用点的 size 参数下界推断"不变量，证据为静态符号 + 反汇编，在静态阶段判定；该优化路径与 size 静默放宽的位置见图 7。

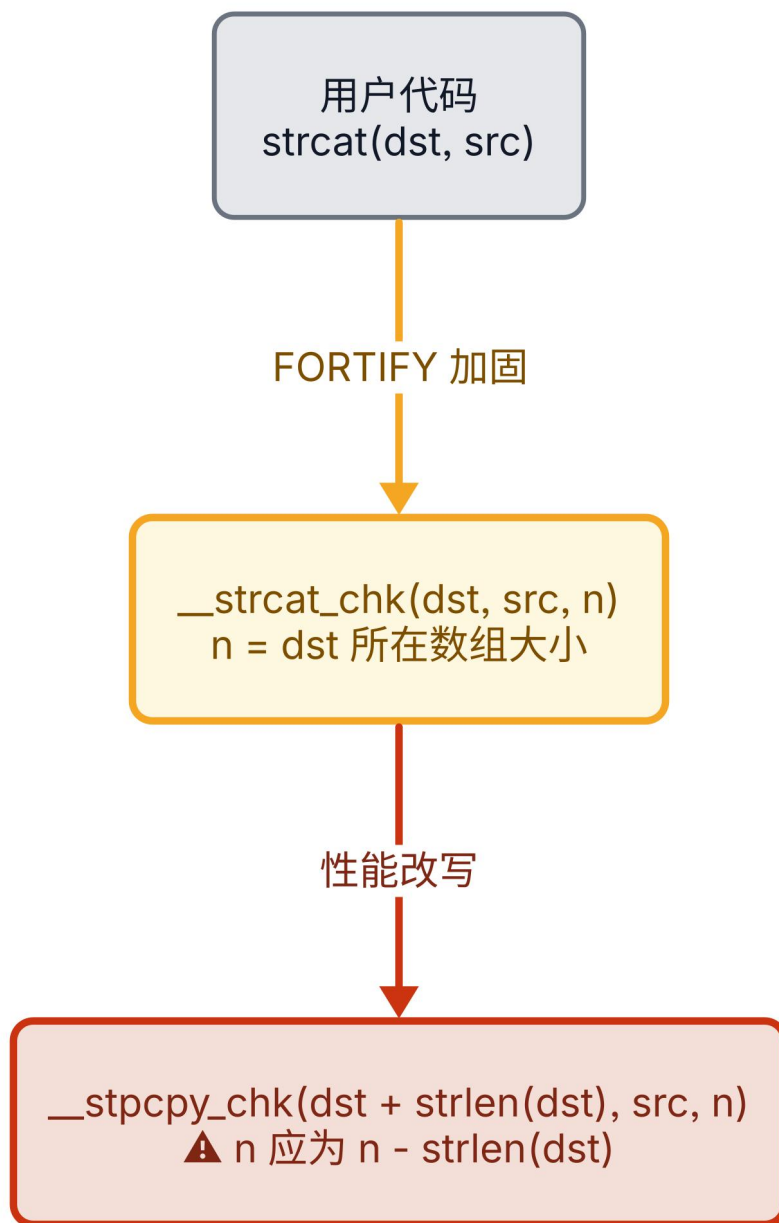


图 7 `_FORTIFY_SOURCE` 优化路径与 `size` 静默放宽

5.2 案例二：IBT 检查漏过 `endbr` 字节模式导致间接跳转保护被绕过

IBT (Indirect Branch Tracking) 是 Intel CET 中针对间接跳转的保护机制：硬件要求间接跳转或调用的目标地址必须以 `endbr` 指令开头，否则触发异常。`endbr` 在二进制中是固定的四字节值 (`endbr64 = F3 0F 1E FA`、`endbr32 = F3 0F 1E FB`)，按设计只应作为合法间接跳

转入口出现在代码段。如果这四个字节以其他形式出现——比如作为 `movabsq` 指令的 64 位立即数的一部分嵌入 `.text`——攻击者就可以把控制流劫持到该处，硬件会将其识别为合法 landing pad，IBT 语义被绕过。

GCC 在 x86 后端用 `ix86_endbr_immediate_operand` 谓词做字节级检测：当目标启用 IBT 时，谓词扫描立即数中是否存在 `endbr` 模式，含该模式的立即数会被拒绝（同源历史修复参见 PR-96350[8]）。本方法在该谓词实现中发现一个逻辑漏洞：扫描循环对“高位非零”的 64 位立即数提前退出，只要 `endbr` 字节排布在立即数高位，扫描就走不到对应位置。这类立即数被错误接受，进而在 `.text` 中嵌入了不受控的伪 landing pad。该缺陷已上报上游（PR-125084[18]），等待维护者回复。

预言机方面，本例命中“`endbr` 字节模式不应出现在非跳转目标的指令中”不变量，证据为代码段内 `endbr` 模式扫描，属静态指令流检查。

5.3 案例三：多 ISA 下 canary 寄存器返回时未清零

本方法在 GCC 通用栈保护后端的 fallback 路径上还找到一例同类失效：MIPS、LoongArch、Xtensa、CSKY 等 ISA 的 fallback 后端未提供专用 epilogue 模板，函数返回时金丝雀寄存器未显式清零，进程级 `__stack_chk_guard` 值在调用者的可见寄存器中残留。该问题与 2020 年针对 AArch64 / ARM 的 PR-96191[7] 同源，当年修复未覆盖上述后端。本课题以 meta-bug PR-125045[19] 统一跟踪该家族在多 ISA 上的实例，其中 LoongArch 子例（PR-125049[20]）已被维护者确认并合入修复，其余 ISA 子例正在依次上报。

5.4 小结

三例分别覆盖静态符号、动态行为、指令流字节模式三类不变量形态，共同特征是：二进制看起来带有保护，但编译器代码路径某一处的安全不变量已失去维护。方法在不修改编译器的前提下，通过约束求解与不变量预言机组合把这类差异在自动化流程中暴露出来。LLVM 侧实验预计在后续阶段以同一预言机框架补全。

参考文献

- [1] Yang X, Chen Y, Eide E, et al. Finding and understanding bugs in C compilers[C]//Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2011: 283-294.
- [2] Livinskii V, Babokin D, Regehr J. Random testing for C and C++ compilers with YARPGen[C]//Proceedings of the ACM on Programming Languages (OOPSLA). 2020, 4: 1-25.
- [3] Zalewski M. American Fuzzy Lop (AFL)[EB/OL]. (2014)[2026-02-15].

<https://lcamtuf.coredump.cx/afl/>.

[4] Serebryany K. Continuous fuzzing with libFuzzer and AddressSanitizer[C]//IEEE Cybersecurity Development (SecDev). 2016: 157.

[5] Yang C, Deng Y, Lu R, et al. WhiteFox: White-box compiler fuzzing empowered by large language models[C]//Proceedings of the ACM on Programming Languages (OOPSLA). 2024.

[6] GNU Project. GCC Bugzilla PR 87672 — [9 regression] 81512c36 causes ICE in bootstrap stage 3 using "-D_FORTIFY_SOURCE=2"[EB/OL]. (2018)[2026-05-11].

https://gcc.gnu.org/bugzilla/show_bug.cgi?id=87672.

[7] GNU Project. GCC Bugzilla PR 96191 — aarch64 stack_protect_test canary leak[EB/OL]. (2020)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191.

[8] GNU Project. GCC Bugzilla PR 96350 — [cet] For ENDBR immediate, the binary would include a gadget that starts with a fake ENDBR64 opcode[EB/OL]. (2020)[2026-05-11].

https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96350.

[9] Intel Corporation. Control-flow Enforcement Technology Specification[R/OL]. (2019)[2026-02-15]. <https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html>.

[10] Free Software Foundation. The GNU C Library Reference Manual: Source Fortification[EB/OL]. (2024)[2026-02-15].

https://sourceware.org/glibc/manual/latest/html_node/Source-Fortification.html.

[11] Le V, Afshari M, Su Z. Compiler validation via equivalence modulo inputs[C]//Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2014: 216-226.

[12] Li S, Su Z. UBfuzz: Finding bugs in sanitizer implementations[C]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024.

[13] Ma H, Shen Q, Tian Y, et al. Fuzzing deep learning compilers with HirGen[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). 2023.

[14] Even-Mendoza K, Sharma A, Donaldson A F, et al. GrayC: Greybox fuzzing of compilers and analysers for C[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software

Testing and Analysis (ISSTA). 2023: 1219-1231.

[15] Yang Y, Zhang H, Ren Z, et al. Hybrid language processor fuzzing via LLM-based constraint solving[C]//Proceedings of the 34th USENIX Security Symposium (USENIX Security). 2025.

[16] Meta Security. CVE-2023-4039: GCC's -fstack-protector fails to guard dynamic stack allocations on AArch64[EB/OL]. (2023)[2026-02-15].

<https://rtx.meta.security/mitigation/2023/09/12/CVE-2023-4039.html>.

[17] GNU Project. GCC Bugzilla PR 125079 — [13/14/15 Regression] tree-ssa-strlen get_string_length: __strcat_chk -> __stpcpy_chk drops bound subtract[EB/OL]. (2026)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079.

[18] GNU Project. GCC Bugzilla PR 125084 — [target/i386] ix86_endbr_immediate_operand misses ENDBR pattern when upper bytes nonzero[EB/OL]. (2026)[2026-05-11].

https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125084.

[19] GNU Project. GCC Bugzilla PR 125045 — [meta-bug] SSP fallback leaves canary in live GPR on many targets[EB/OL]. (2026)[2026-05-11].

https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125045.

[20] GNU Project. GCC Bugzilla PR 125049 — [loongarch] SSP fallback leaves __stack_chk_guard live in \$r12 at `jr \$r1`[EB/OL]. (2026)[2026-05-11].

https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125049.

二、学位论文工作实施计划

（一）论文的理论分析与硬件要求及其预期达到的水平与结果

理论分析涉及三方面：编译器主流防御机制的安全不变量及其证据形式、覆盖率与控制流图引导的目标基本块选择策略、大模型生成能力与预言机判定的耦合方式。预期产出包括：一套覆盖至少三种主流防御机制的不变量集合、一套可扩展的预言机框架、一个可面向 GCC 与 LLVM 跨架构运行的 fuzzer 原型；通过在 GCC 与 LLVM 主流版本上发现真实 silent failure 来验证有效性。硬件方面，开发与实验依托一台 64 位 Linux 工作站（不少于 32 GB 内存、百 GB 级 SSD），用于 GCC / LLVM 多版本本地构建、覆盖率数据缓存与跨架构 QEMU 用户态执行；大模型推理通过远程 API 完成，本机不需要独立 GPU。

（二）论文工作进度与安排

起讫日期	工 作 内 容 和 要 求	备 注
2026.05-2026.07	扩展不变量集合到 BTI / PAC、Shadow Stack、CFI 等机制，丰富预言机框架的静态与动态检查器	
2026.07-2026.09	把编译器子系统、覆盖率适配层、flag profile 扩展到 LLVM 工具链，对齐跨架构 / 跨编译器实验框架	
2026.09-2026.12	在 GCC 与 LLVM 多版本、多 ISA 上完成全机制端到端实验，持续推进上游缺陷上报闭环	
2026.12-2027.02	与 Csmith、YARPGen、GrayC、HLPFuzz 等基线方法做对照评测，量化覆盖率推进与缺陷发现上的相对优势	
2027.02-2027.04	整理实验数据与图表，完成学位论文初稿，准备会议 / 期刊投稿	
2027.04-2027.06	学位论文修订、外审反馈处理、答辩材料准备	
2027.06	学位论文答辩	
学校指导教师对 开题报告的综合 意见	指导教师 (签字) 年 月 日	
校外指导 教师对开题报告 的综合意见		

备注：	